Arrays and Plots: Represent a physical model and compare prediction with evidence

LECTURE DEMONSTRATION

PHY4605 Physics-First Reproducible MATLAB Notebook

Physical Question and Prediction

A ball is launched straight upward. Before computing, predict the shape of its vertical-position graph during the first four seconds: it should rise, slow to a highest point, then fall.

Learning outcomes. By the end of this demonstration, you can identify variables and units in a familiar model, evaluate that model over an array, and read a labelled plot with one validation check.

Model, Assumptions, and Units

We choose upward as positive. Air resistance is neglected and gravity is constant. In words: position equals the starting position plus the upward launch contribution minus the downward gravity contribution.

The analytical model is $y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$. Here `y_m` is position in m, `t_s` is time in s, `y0_m` is the starting position in m, `v0_mps` is launch velocity in m s^-1, and `g_mps2` is gravitational acceleration in m s^-2.

This code assigns meaningful scalar parameters. Read the units in the variable names before running the section.

```
y0_m = 0;          % starting vertical position (m)
v0_mps = 20;       % upward launch velocity (m s^-1)
g_mps2 = 9.81;     % gravitational acceleration magnitude (m s^-2)
```

Time as an Array

A scalar holds one value; an array holds many values. The colon expression below starts at 0 s, advances by 0.1 s, and ends at 4 s. MATLAB indexing starts at 1, so `t_s(1)` is the first stored time.

Run this section and inspect the three displayed values: the array length, its first value, and its eleventh value.

```
t_s = 0:0.1:4;     % time samples from 0 s to 4 s
n_samples = length(t_s)
```

```
n_samples =
41
```

```
first_time_s = t_s(1)
```

```
first_time_s =
0
```

```
eleventh_time_s = t_s(11)
```

```
eleventh_time_s =
1
```

Algorithm Before Code

Pseudocode: (1) choose the physical parameters; (2) make an array of times; (3) calculate position at every time; (4) plot position against time; (5) check the known starting value; (6) explain the graph.

In the equation, `*`, `/`, and `^` are for scalar operations. Because `t_s` is an array, MATLAB needs `.*`, `./`, and `.^` when an operation is applied element by element. The dot means: do this to every time sample.

Evaluate the Model at Every Time Sample

This short calculation applies the familiar equation to all 41 entries of `t_s`. The two dots are essential: `.*` multiplies each time sample by `v0_mps`, and `.^2` squares each time sample.

```
y_m = y0_m + v0_mps.*t_s - 0.5*g_mps2.*t_s.^2;
position_at_start_m = y_m(1)
```

```
position_at_start_m =
0
```

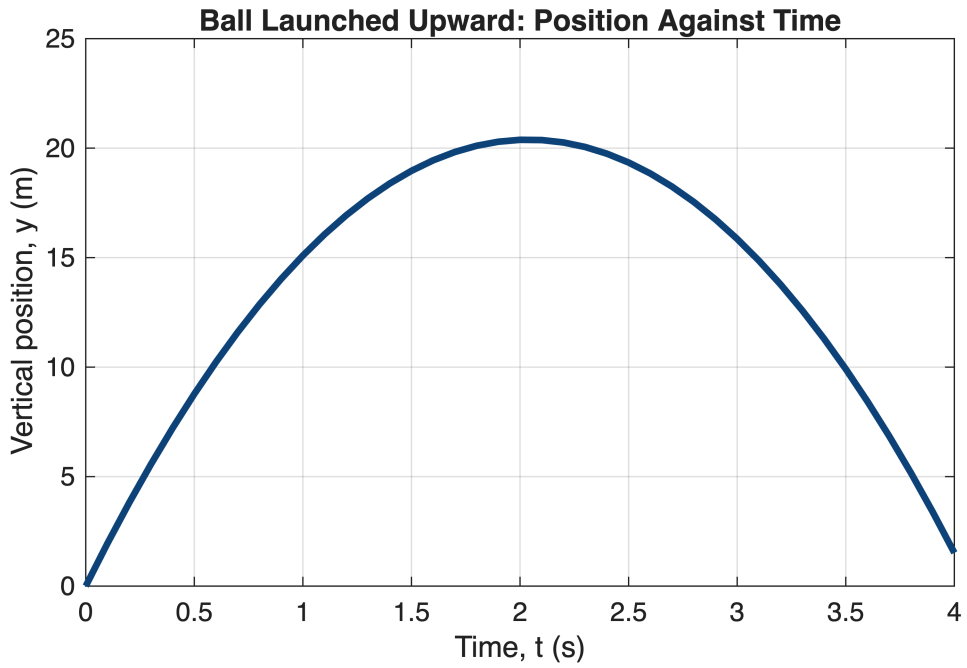
```
position_at_1s_m = y_m(11)
```

```
position_at_1s_m =
15.0950
```

Plot the Physical Output

The horizontal axis is time in s; the vertical axis is vertical position in m. Run the section, then compare the plotted curve with your prediction before computing.

```
plot(t_s,y_m,'LineWidth',2.5,'Color',[0.05 0.26 0.47])
grid on
xlabel('Time, t (s)')
ylabel('Vertical position, y (m)')
title('Ball Launched Upward: Position Against Time')
```



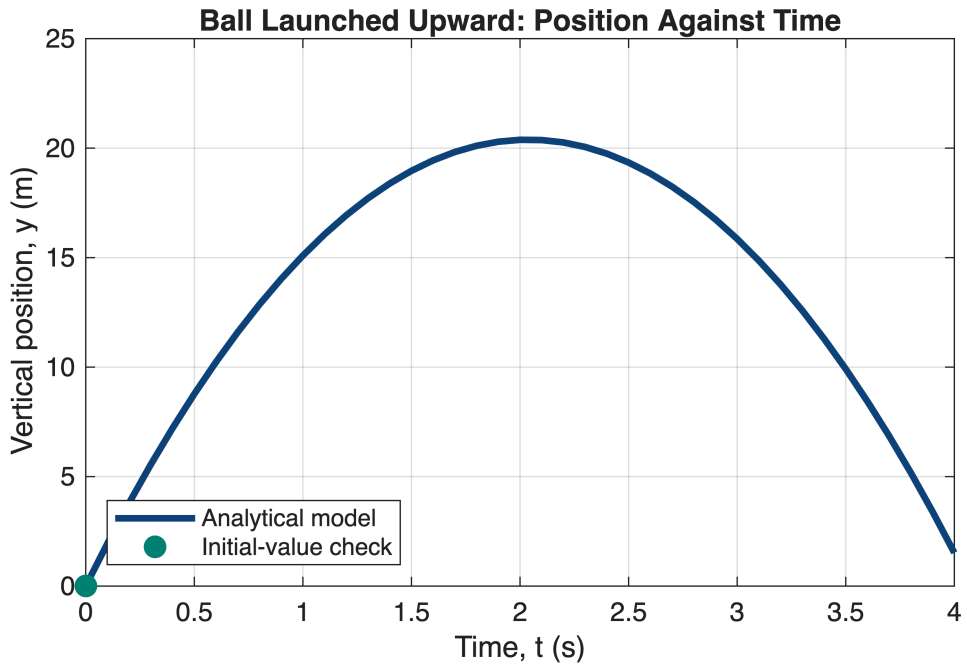
Core Validation: Check the Known Starting Value

At `t=0`, the model must return the stated starting position, 0 m. This executable assertion checks that claim using a tiny numerical tolerance. If it runs without an error, the check passes.

```
assert(abs(y_m(1)-y0_m) < 1e-12,'Initial-position validation failed.')
```

The teal marker below makes the same check visible on the graph.

```
hold on
plot(t_s(1),y_m(1),'o','MarkerSize',8,'MarkerFaceColor',[0.00 0.50 0.45], ...
   'MarkerEdgeColor',[0.00 0.50 0.45])
legend('Analytical model','Initial-value check','Location','southwest')
```



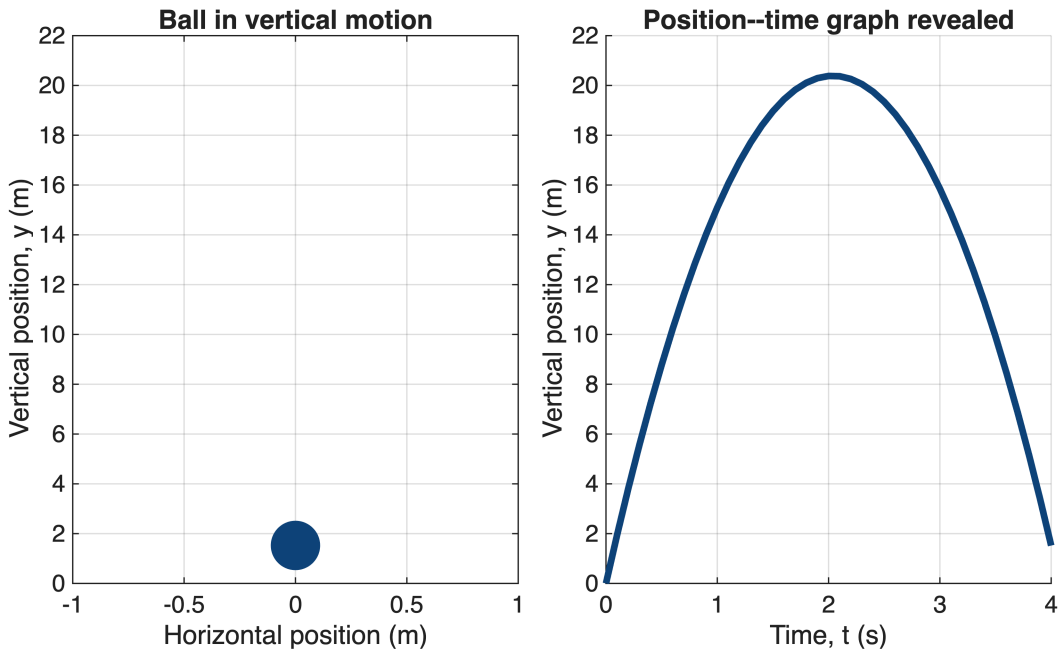
Working Exposure: Animate the Same Model

This lecturer-controlled animation shows the ball moving upward and downward while the position--time curve is revealed. The supplied `for` loop is only a display tool: writing loops is introduced as Core work in Week 2. The model, array, and validation are unchanged.

Run this section after the graph section. Watch how each moving-ball position corresponds to one new point on the graph.

```
tiledlayout(1,2,'TileSpacing','compact','Padding','compact')
nexttile
ball_handle = plot(0,y_m(1),'o','MarkerSize',18,'MarkerFaceColor',[0.05 0.26
0.47], ...
   'MarkerEdgeColor',[0.05 0.26 0.47]);
grid on
xlim([-1 1])
ylim([0 22])
xlabel('Horizontal position (m)')
ylabel('Vertical position, y (m)')
title('Ball in vertical motion')
nexttile
trace_handle = animatedline('Color',[0.05 0.26 0.47],'LineWidth',2.5);
grid on
xlim([0 4])
ylim([0 22])
xlabel('Time, t (s)')
ylabel('Vertical position, y (m)')
title('Position--time graph revealed')
for sample_index = 1:numel(t_s)
```

```
    set(ball_handle,'YData',y_m(sample_index))
    addpoints(trace_handle,t_s(sample_index),y_m(sample_index))
    drawnow
    pause(0.04)
end
```



Interpret the Result

The curve rises because the initial velocity is upward. It bends downward because gravity reduces the upward velocity, reaches a highest point near 2.04 s, then returns downward. The equation, array calculation, and graph describe the same physical motion.

Working Exposure: Selected Values and Continuous Reference

This optional section shows a compact table and the continuous analytical peak. It is useful evidence, but the Core route is already complete without it.

```
selected = table(t_s([1 11 21 31 41])',y_m([1 11 21 31 41])', ...
    'VariableNames',{'time_s','position_m'})
```

`selected` = 5×2 table

	time_s	position_m
1	0	0
2	1	15.0950
3	2	20.3800
4	3	15.8550

	time_s	position_m
5	4	1.5200

```
t_peak_s = v0_mps/g_mps2
```

```
t_peak_s =
2.0387
```

```
y_peak_m = y0_m + v0_mps^2/(2*g_mps2)
```

```
y_peak_m =
20.3874
```

Reproducibility Record

Run sections from top to bottom in a fresh MATLAB session. This demonstration requires MATLAB R2025a or later and base MATLAB only. Locked inputs: `y0_m=0 m`, `v0_mps=20 m s^-1`, `g_mps2=9.81 m s^-2`, and `t_s=0:0.1:4 s`.